



Composables with Jetpack

IS2205 - Mobile Application Design & Development

Agenda

- 
- ❏ From XML markup To composable components
 - ❏ Structural containers from composables
 - ❏ Styling properties
 - ❏ Previews
 - ❏ Event Handling
 - ❏ Separating View & Control

Composables

Jetpack Compose is a modern UI library for Android UI development

It uses a set of composable functions as basic building blocks of a UI

A composable function:

- Describes some part of your UI
- Does **not** return anything
- Takes inputs and based on, generates components on screen

Imperative vs. Declarative

- 
- ❑ XML based approach: Mutating an active tree of View components via methods like `findViewById()` and `setText()` programmatically.
 - ❑ Compose approach: Formulating the UI dynamically as an immutable function that updates automatically whenever data changes.

Imperative...

1. `<TextView`

2. `android:id="@+id/statusText"`

3. `android:layout_width="wrap_content"`

4. `android:layout_height="wrap_content"`

5. `android:text="Initial Text" />`

6.

7. `val textView =`

`findViewById<TextView>(R.id.statusText)`

8. `textView.text = "Updated Text"`

Annotations

Composables use Annotations attaching extra information (metadata) to the code

The Jetpack Compose compiler understand the specific annotated code

Declared by prefixing its name with an @ character at the beginning

Code elements such as properties, functions, and classes, can be annotated i.e.:

@myannotation

```
fun myFunction(){}  

```

Annotations in Kotlin



Annotating as a
Composable

Prefix character: @

Annotation

@Composable

```
fun Greeting(name: String, modifier: Modifier) {}
```

Function declaration

Declarative...

1. @Composable
2. fun StatusDisplay(text: String) {
3. Text(text = text)
4. }
- 5.

Access to the component is declared via an annotation

Calls Text inside it which is another composable

View Components vs. Composables

- 
- ❑ New Building Blocks: Transforming standalone XML nodes (TextView, ImageView, Button) into Kotlin standalone function invocations annotated with `@Composable`.
 - ❑ Best Practices & Naming Conventions: Nouns using PascalCase (CustomButton) instead of camelCase verbs (renderButton).

Companion Note

View Markup vs. Composable Functions

1. <Button
2. android:layout_width="wrap_content"
3. android:layout_height="wrap_content"
4. android:text="Submit" />
- 5.
6. @Composable
7. fun SubmitButton() {
8. Button(onClick = { /* Action */ }) {
9. Text(text = "Submit")
10. }
11. }

Structural Containers

- 
- ❏ ViewGroups : the traditional View system, layout structures with nested multi-layered XML elements.
 - ❏ Structural Composables : layout containers are lightweight Kotlin inline functions that receive UI children as a trailing lambda block (content = { ... }).

Structural Containers

- ❏ `LinearLayout : android:orientation="vertical" =>`
`Column Composable : Elements are arranged in vertical axis.`
- ❏ `LinearLayout : android:orientation="horizontal" =>` `Row Composable : Elements are arranged in horizontal axis`

Structural Containers...

- ❑ `FrameLayout` : Frame/Stack views on top of each other (overlapping z-axis children), OR as an anchor container for fragments => Jetpack Box : The last element declared inside the Box block renders on top.
- ❑ Relative/Complex Boundaries: `ConstraintLayout` => `ConstraintLayout` (in Compose Library) : Instead of using complex nested structural limits, both approaches use constraints to flatten layout trees.

View Group : Linear Vertical

1. <LinearLayout
2. ...
3. android:orientation="vertical">
4. <TextView
5. ...
6. android:text="Title" />
- 7.
8. <TextView
9. ...
10. android:text="Subtitle" />
11. </LinearLayout>

Structural Composables : Column

```
1. @Composable
2. fun VerticalLayoutSample() {
3.     Column(
4.         modifier = Modifier
5.             .fillMaxWidth()
6.             .padding(16.dp)
7.     ){
8.         Text(text = "Title")
9.         Text(text = "Subtitle")
10.    }
11. }
```

View Group : Linear Horizontal

1. <LinearLayout
2. ...
3. android:orientation="horizontal">
4. <TextView
5. ...
6. android:text="Title" />
- 7.
8. <TextView
9. ...
10. android:text="Subtitle" />
11. </LinearLayout>

Structural Composables : Row

```
1. @Composable
2. fun HorizontalLayoutSample() {
3.     Row(
4.         modifier = Modifier.fillMaxWidth(),
5.         verticalAlignment = Alignment.CenterVertically
6.     ){
7.         Text(text = "Title")
8.         Text(text = "Subtitle")
9.     }
10. }
```

View Group : FrameLayout

```
1. <FrameLayout... >
2. <ImageView
3.     ...
4.     android:src="@drawable/background_image"
5. />
6.
7. <TextView
8.     ...
9.     android:text="Overlay Text"
10.    android:textColor="#FFFFFF" />
11. </FrameLayout>
```

Structural Composables : Box

```
1. @Composable
2. fun StackingLayoutSample() {
3.     Box(modifier = Modifier.fillMaxWidth().height(200.dp)
4.         ){
5.         Image(painter = painterResource(id =
6.             R.drawable.background_image), ... )
7.         Text(text = "Overlay Text", color = Color.White, ...
8.             )
9.     }
```

View Groups vs Structural Composables

Constraint Layout in Two approaches



=> Companion Note

Styling Properties

View Attributes vs. Modifiers

- ❑ The Decorator Pattern: Replacing inline XML configuration strings (android:padding, android:layout_width) with chained Modifier instances.
- ❑ Ordering Mechanics: Modifiers process operations strictly sequentially => configuring padding before a background may produce a different boundary effect than background before padding.

Modifiers...

❏ The Decorator Pattern : The Modifier Chain

Classic Java/Groovy Builder Pattern: Mutate and return the exact same object reference (this).

❏ Compose Modifier Chain: Build an immutable linked list. Each extension function creates a new wrapper object that encloses the previous one .

Integrated Previews

XML Preview vs. @Preview Approach

- ❑ Design Previews: Design View IDE tool with static  Layout design viewers.
- ❑ Dynamic Previews: Leveraging the @Preview compiler tag annotation and parameters (showBackground, showSystemUi) directly alongside the actual layout definition.

Layout Composables



1. `@Preview(showBackground = true, showSystemUi = true)`
2. `@Composable`
3. `fun ScreenMaking() {`
4. `VerticalStack()`
5. `}`

Event Handling

Definitions vs. Lambda State Hoisting

- ❑ XML Approach: Attaching a string method pointer in XML attributes or manually setting a listener class instance (setOnClickListener) on an existing element reference
- ❑ Compose Approach (State Hoisting): Composable UI elements do not hold or directly modify data themselves.
- ❑ They accept a data state value as a parameter and expose execution events upward as Kotlin function types (lambdas) to be handled by a parent controller.

Layout Markup

1. <Button ...
2.  android:onClick="onSaveButtonClick" />
- 3.
4. //Kotlin
5. fun onSaveButtonClick(view: View) {
6. // **view** refers to the button
7. Toast.makeText(this, "Saved!",
 Toast.LENGTH_SHORT).show()
8. }
- 9.

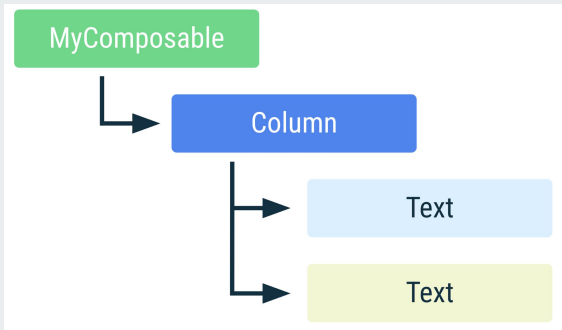
Layout Markup

1. // The View is decoupled from control logic
2. // and reusable
3. @Composable
4. fun ActionButton(buttonText: String,
onExecuteAction: () -> Unit) { //refer the function
5. Button(onClick = onExecuteAction) { //use it for
6. Text(text = buttonText) //event handling
7. }
8. }
- 9.

More...



Sample Code



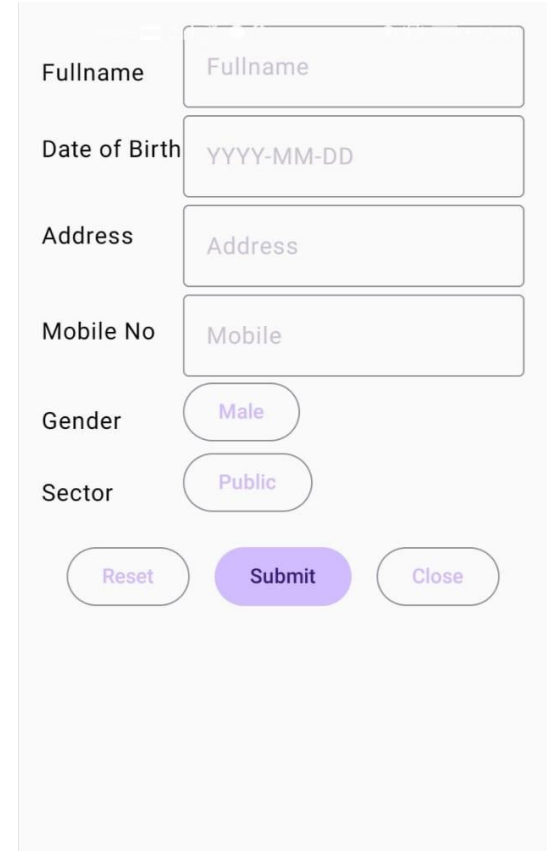
Images from:

<https://developer.android.com/develop/ui/compose/lifecycle>

<https://android-developers.googleblog.com/2019/05/whats-new-with-android-jetpack.html>

Activities

1. Create the default greeting application in  Android Studio. By using horizontal and vertical movements reposition the component center horizontally.
2. Try to build a UI similar to the one in the right hand side



Registration

Fullname

Date of Birth YYYY-MM-DD

Address

Mobile No Mobile

Gender Male

Sector Public

Reset Submit Close

The End